
HOSPITAL EMERGENCY QUEUE MANAGEMENT SYSTEM USING PRIORITY QUEUE

Annamacharya Institute Of Technology And Sciences Tirupathi

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2025 – 2027

PROJECT REPORT

Submitted By

T. C. VARALAKSHMI(23AK1A05M4)

B.Tech(CSE)

AITS-Tirupathi

Under the Guidance of

Training And Placement Officer

Academic Year

2025 – 2027

CERTIFICATE

This is to certify that the project entitled "Hospital Emergency Queue Management System Using Priority Queue" submitted by T. C. Varalakshmi(23AK1A05M4)in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering is a bonafide work carried out during the academic year 2025–2027 under our supervision.

The work embodied in this project has not been submitted previously for the award of any degree or diploma.

Head of Department: Dr. T.SREENIVASULA REDDY, M.Tech,Ph.D

HOD & Professor

Department of CSE

Date : 25-06-2026

Place : Tirupathi

DECLARATION

I hereby declare that the project work entitled "Hospital Emergency Queue Management System Using Priority Queue" is an original work carried out by me under the guidance of the faculty guide.

The work presented in this report has not been submitted elsewhere for the award of any degree or diploma.

Student Signature

T. C. Varalakshmi (23AK1A05M4)

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Head of Department, faculty members, and institution for providing continuous support and guidance throughout the development of this project.

I would also like to thank my family and friends for their encouragement and motivation during the successful completion of this work.

ABSTRACT

Hospital emergency departments are responsible for providing immediate medical attention to patients with varying levels of urgency. Traditional first-come-first-served approaches are often inadequate because they do not consider the severity of a patient's condition.

The Hospital Emergency Queue Management System Using Priority Queue addresses this challenge by implementing a Max Heap-based Priority Queue that automatically prioritizes patients according to their medical severity. The system integrates Flask REST APIs, PostgreSQL database storage, and Google Gemini AI to provide intelligent severity recommendations and efficient queue management.

The proposed solution ensures that critical patients receive timely treatment, reduces waiting times, improves operational efficiency, and enhances decision-making capabilities in emergency departments.

Keywords:

Priority Queue, Max Heap, Flask, PostgreSQL, Gemini AI, Emergency Management System, Healthcare Technology.

TABLE OF CONTENTS

Certificate	
Declaration	
Acknowledgement	
Abstract	
Chapter 1 – Introduction	
Chapter 2 – Literature Survey	
Chapter 3 – System Analysis	
Chapter 4 – System Design	
Chapter 5 – Technologies Used	
Chapter 6 – Implementation	
Chapter 7 – Working of the System	
Chapter 8 – Testing and Validation	
Chapter 9 – Advantages and Applications	
Chapter 10 – Future Enhancements	
Chapter 11 – Complexity Analysis	
Chapter 12 – Screenshots and Results	
Chapter 13 – Conclusion	
References	
Appendix	

CHAPTER 1

INTRODUCTION

1.1 Background

Healthcare organizations continuously face challenges in managing emergency patients effectively. Emergency departments often experience high patient volumes, requiring immediate decisions regarding patient treatment priorities.

In many hospitals, emergency patient handling is performed manually. This process can lead to delays, inconsistencies, and inefficiencies, especially during peak hours when multiple critical patients arrive simultaneously.

A Priority Queue-based management system provides an effective solution by automatically organizing patients according to their severity levels. This ensures that critically ill patients receive medical attention before less severe cases.

1.2 Problem Statement

Emergency rooms frequently encounter patients with varying levels of medical urgency. Traditional first-come-first-served approaches fail to prioritize patients based on medical severity, resulting in delayed treatment for critically ill patients.

Manual prioritization processes are susceptible to human errors, inefficient resource utilization, and increased waiting times. Therefore, there is a need for an intelligent and automated emergency queue management system capable of prioritizing patients effectively.

1.3 Objectives

The primary objectives of this project are:

- To prioritize patients according to medical severity.
- To reduce waiting times in emergency departments.
- To improve treatment efficiency and patient outcomes.
- To provide AI-assisted severity recommendations.
- To maintain persistent patient records.
- To automate emergency patient management workflows.

1.4 Scope of the Project

The project focuses on emergency patient management in hospitals and healthcare centers. It includes:

-
-
- Patient registration.
 - Severity assessment.
 - AI-assisted triage.
 - Priority Queue management.
 - Patient admission.
 - Database storage.
 - Reporting and statistics.

1.5 Need for the Project

The increasing demand for emergency healthcare services requires efficient patient management systems. An automated prioritization mechanism helps hospitals deliver timely medical care, reduce waiting times, and improve patient satisfaction.

1.6 Project Overview

The Hospital Emergency Queue Management System is developed using Python, Flask, PostgreSQL, and Google Gemini AI. The system employs a Max Heap Priority Queue to ensure that patients with the highest severity levels receive treatment first.

The project demonstrates the practical application of Data Structures and Algorithms in solving real-world healthcare problems.

1.7 Expected Outcomes

- Improved emergency response.
- Better resource utilization.
- Reduced patient waiting time.
- Enhanced decision-making.
- Reliable record management.
- Scalable healthcare solution.

CHAPTER 2 –

LITERATURE SURVEY

Related Studies

Several healthcare management systems have been developed to improve patient care. Most existing systems focus on record management rather than emergency prioritization. Priority Queue-based systems have shown significant improvements in reducing waiting times and improving patient outcomes.

Research Gap

Most traditional systems do not integrate AI-based severity assessment and efficient queue management together. This project bridges that gap.

CHAPTER 3 –

SYSTEM ANALYSIS

Feasibility Study

Technical Feasibility

The project uses modern technologies such as Python, Flask, PostgreSQL, and Gemini AI which are widely supported.

Economic Feasibility

The project uses open-source technologies, reducing implementation cost.

Operational Feasibility

The system is easy to use and requires minimal training.

CHAPTER 4 – SYSTEM DESIGN

Sequence Diagram Description

Receptionist → System → AI Engine → Priority Queue → Doctor

1. Patient information entered.
2. AI analyzes symptoms.
3. Severity score generated.
4. Patient inserted into queue.
5. Doctor treats highest-priority patient.

ER Diagram Components

Entities:

- Patient
- Queue
- Doctor
- Audit Log
- User

Relationships:

- Doctor treats Patient.
- Patient belongs to Queue.
- Audit Log tracks activities.

CHAPTER 5 – TECHNOLOGIES USED

Python

Python was selected due to its simplicity, extensive libraries, and strong backend capabilities.

Flask

Flask provides lightweight REST API development and rapid deployment.

PostgreSQL

PostgreSQL ensures reliable and scalable data storage.

Google Gemini AI

Gemini AI helps in analyzing symptoms and suggesting patient severity levels.

CHAPTER 6 – IMPLEMENTATION

Module 1: Patient Registration

Stores patient details including:

- Name
- Age
- Gender
- Symptoms
- Contact Information

Module 2: AI Severity Assessment

Analyzes symptoms and recommends priority.

Module 3: Queue Management

Uses Max Heap to maintain patient order.

Module 4: Reporting

Generates queue statistics and patient reports.

CHAPTER 7 – WORKING OF THE SYSTEM

Example Scenario

Patients:

Patient Severity

A 4

B 9

C 7

Priority Queue Order:

B (9)

C (7)

A (4)

Patient B receives treatment first because of highest severity.

CHAPTER 8 – TESTING

Testing Objectives

- Verify correctness.
- Ensure reliability.
- Validate API responses.
- Check queue operations.

Sample Test Cases

Test ID	Description	Expected Result
TC01	Add Patient	Success
TC02	Search Patient	Found
TC03	Update Priority	Updated
TC04	Admit Patient	Highest Priority Returned

CHAPTER 9 – ADVANTAGES

Organizational Benefits

- Improved patient satisfaction.
- Reduced workload for staff.
- Better emergency handling.

Technical Benefits

- Fast heap operations.
 - Reliable database storage.
 - AI-assisted decisions.
-

CHAPTER 10 – FUTURE ENHANCEMENTS

Advanced Features

- Mobile application.
- Doctor dashboard.
- Predictive analytics.
- Cloud deployment.
- Voice-based symptom entry.
- Integration with wearable devices.

CHAPTER 11 – COMPLEXITY ANALYSIS

Space Complexity

Operation	Complexity
Storage	$O(n)$

Time Complexity

Operation	Complexity
Insert	$O(\log n)$
Delete Max	$O(\log n)$
Peek	$O(1)$
Search	$O(n)$

CHAPTER 12 – SCREENSHOTS AND RESULTS

Add a full page for each screenshot:

1. Landing Page
 2. Patient Registration Page
 3. AI Recommendation Screen
 4. Queue Dashboard
 5. Admit Patient Screen
 6. Database Records
 7. Test Execution Results
-

CHAPTER 13 – CONCLUSION

Add one extra paragraph:

The project successfully demonstrates how Data Structures can solve real-world healthcare challenges. By integrating Priority Queues, AI-powered severity analysis, and database persistence, the system delivers a scalable and intelligent emergency management solution suitable for modern hospitals.

